Full Stack Assignment – 40

Assignment ID: FSD-40

Title: Conditional Escalation Management System

Duration: 3 Hours

1️⃣ Objective

Design and develop a **Conditional Escalation Management System** that allows issues to be raised and automatically escalated based on predefined conditions.
The system must enforce escalation rules and ensure **data persistence using a database**.

2️⃣ User Roles

REPORTER

- Register and log in to the system
- Raise issues with severity level
- View own issue status

HANDLER

- Register and log in to the system
- Act on assigned issues
- Update issue status

ESCALATION_MANAGER

- Register and log in to the system
- View escalated issues
- Resolve escalated issues

3️⃣ Functional Requirements

Authentication & Authorization

- User registration and login
- **JWT-based authentication**
- Protected backend APIs
- Role-based access control

Issue Management

- REPORTER submits issues with severity
- Issues must be stored in the database
- HANDLER updates issue status
- Issue state must be retrieved dynamically

Escalation Logic

- Issues escalate automatically when:
  - Severity exceeds a threshold OR
  - Issue remains unresolved beyond a defined time
- Escalation state must be stored in the database

4️ Business Rule (Critical)

⚠️ **Once an issue is escalated, only an ESCALATION_MANAGER can resolve it.**

Handlers must not act on escalated issues.

5️ Technical Constraints (Mandatory)

- All issue and escalation data **must persist in a database**
- No hardcoded or in-memory escalation logic
- JWT authentication and role checks mandatory
- Environment variables required

6️ Frontend Requirements

- Role-based dashboards for REPORTER, HANDLER, ESCALATION_MANAGER
- Issue creation form for REPORTER
- Issue handling view for HANDLER
- Escalated issue resolution view for ESCALATION_MANAGER

UI is secondary; escalation correctness is primary.

7️ Submission Requirements (Strict)

GitHub Repository

- Backend source code
- Frontend source code
- Proper folder structure
- Clear and meaningful commit history

Live Deployment

- Live frontend deployment link
- Live backend deployment link
- Backend must be connected to the database

README.md (Mandatory)

The README file must include:

- Project overview
- Tech stack used
- User roles and permissions
- API endpoints
- Database schema or collections
- Steps to run the project locally
- Live frontend and backend links

8️⃣ Evaluation Note

Focus on **conditional logic, role transitions, and data integrity**.